

EOS Composition Partial: Implementation Map

Skye, eosDT, Brunt, and implicit diffusion follow-up

2026-05-14

This document is a rendered companion to `notes/eos_composition_partials_plan.md`. It separates what MESA already does from what must be added, and it flags implementation effort. No MESA compile, MESA run, test suite, or EOS table regeneration has been done.

Legend

Current MESA behavior. Existing code or equations already present in the repository. These are source-level facts to preserve or use.

New EOS implementation. Analytic EOS-side derivatives to add, mostly in `eos/private`. This is the main work needed to remove finite-difference composition partials for Skye/eosDT-covered cells.

New or changed star-side use. Changes in `star/private` that consume the new EOS partials, especially Brunt and possible coupled implicit diffusion work.

Verification only. Finite differences are acceptable here as tests of analytic derivatives, but not as production replacement partials.

Design decision. A choice that should be made explicitly before implementation broadens.

Risk or nonlinear boundary. Active-set changes, blend kinks, phase branches, and API expansion points.

Effort Summary

Area	Color	Effort	Why
Composition helper and moment derivatives	New EOS	S	Mostly wraps formulas already in <code>chem_lib</code> .
Skye ideal-ion partials	New EOS	M	Direct analytic chain rule through active-set number fractions.
Skye electron partials	New EOS	M	HELM derivative tables already contain the needed density derivatives.
Skye Coulomb partials	New EOS	L/XL	Many scalar composition inputs and phase/branch logic.

Area	Color	Effort	Why
eosDT blend dalpha/dX_i	New EOS	M	Mechanical plumbing plus blend alpha derivatives.
Brunt chain-rule form	Star-side	M	Branch exists, but needs total-pressure conversion and analytic EOS partials.
Coupled implicit diffusion	Star-side	XL	Separate solver-structure project; EOS partials are prerequisite.
Rotation/other transport compatibility	Star-side	M	Default-off controls should preserve current operator-split workflows.
Finite-difference checks	Verification	M	Needed to validate; not production behavior.

Current Contract

EOS public export.

`num_eos_d_dxa_results = 2`

Only the first two composition derivative rows are reliably returned to `star`:

$$d_dxa(i_lnP_{gas}, j) = \left. \frac{\partial \ln P_{gas}}{\partial X_j} \right|_{\rho, T}, \quad d_dxa(i_lnE, j) = \left. \frac{\partial \ln E}{\partial X_j} \right|_{\rho, T}.$$

Internally `eos/public/eos_lib.f90` already allocates `num_eos_basic_results` rows and then copies only the first two rows back to `star`.

Composition normalization and gauge.

MESA composition variables satisfy

$$S_X \equiv \sum_i X_i = 1, \quad \sum_i \delta X_i = 0$$

for physical perturbations. A species-basis derivative vector has gauge freedom: adding a constant to all species partials does not change any constrained derivative or any contraction with a zero-sum gradient.

$$\sum_i \left(\frac{\partial Q}{\partial X_i} + C \right) \delta X_i = \sum_i \frac{\partial Q}{\partial X_i} \delta X_i.$$

`star/private/star_solver.f90` currently uses a sink projection:

$$D_j^{\text{sink}} Q = \frac{\partial Q}{\partial X_j} - \frac{\partial Q}{\partial X_s}.$$

Composition Moments

Existing equations in `chem/public/chem_lib.f90`.

Let

$$Y_i = \frac{X_i}{A_i}, \quad Y = \sum_i Y_i, \quad \bar{A} = \frac{S_X}{Y}.$$

For a charge moment g_i ,

$$M_g = \bar{A} \sum_i \frac{X_i}{A_i} g_i.$$

MESA uses $g_i = Z_i, Z_i^2$, and `chem_isos%Z53` for $\bar{Z}$, $\bar{Z}^2$, and $\bar{Z}^{5/3}$. The implemented derivatives are

$$\begin{aligned} \frac{\partial \bar{A}}{\partial X_j} &= \frac{\bar{A}}{A_j S_X} (A_j - \bar{A}), \\ \frac{\partial M_g}{\partial X_j} &= \frac{\bar{A}}{A_j S_X} (g_j - M_g), \\ m_c &= \sum_i X_i \frac{W_i}{A_i}, \quad \frac{\partial m_c}{\partial X_j} = \frac{W_j}{A_j} - m_c. \end{aligned}$$

Add reusable EOS helper.

Add a private helper, probably `eos/private/eos_composition_partials.f90`, that returns

$$\bar{A}_j, \quad \bar{Z}_j, \quad Y_{e,j}, \quad \bar{Z}^2_j, \quad \bar{Z}^{5/3}_j, \quad m_{c,j},$$

plus active-set normalized number-fraction derivatives for Skye.

For complete ionization,

$$Y_e = \frac{\bar{Z}}{\bar{A}} = \sum_i X_i \frac{Z_i}{A_i}, \quad Y_{e,j} = \frac{Z_j}{A_j} - Y_e.$$

Effort: **S**. These are direct formulas already consistent with `chem_lib`.

Skye Analytic Partial

Current Skye state.

`eos/private/skye.f90:Get_Skye_EOS_Results` calls `sky_eos(..., d_dxa, ierr)` and then sets

$$d_dxa = 0.$$

The value path already uses `auto_diff_real_2var_order3` for T and ρ , but not for arbitrary composition variables.

Core Skye strategy.

Do not introduce global arbitrary-size autodiff. For each species j , compute a companion object

$$F_j = \left. \frac{\partial F}{\partial X_j} \right|_{\rho, T},$$

where F_j itself is still an `auto_diff_real_2var_order3` object in T and ρ .

Skye's existing thermodynamics are

$$S = -\frac{\partial F}{\partial T}, \quad P_{\text{gas}} = \rho^2 \frac{\partial F}{\partial \rho}, \quad E = F + TS.$$

Therefore

$$S_j = -\frac{\partial F_j}{\partial T}, \quad P_{\text{gas},j} = \rho^2 \frac{\partial F_j}{\partial \rho}, \quad E_j = F_j + TS_j.$$

The packed outputs are

$$\frac{\partial \ln S}{\partial X_j} = \frac{S_j}{S}, \quad \frac{\partial \ln E}{\partial X_j} = \frac{E_j}{E}, \quad \frac{\partial \ln P_{\text{gas}}}{\partial X_j} = \frac{P_{\text{gas},j}}{P_{\text{gas}}}.$$

Effort: **M** for the framework, before Coulomb.

Ideal-ion term.

Skye's ideal-ion free energy is

$$F_{\text{ion}} = \frac{k_B T}{m_u \bar{A}} \Phi, \\ \Phi = \sum_i y_i \left[\ln \left(\frac{y_i n}{n_{Q,i}} \right) - 1 \right], \quad n = \frac{\rho}{m_u \bar{A}}, \quad n_{Q,i} = n_Q W_i^{3/2}.$$

With active-set normalized number fractions y_i ,

$$\Phi_j = \sum_i y_{i,j} \ln \left(\frac{y_i n}{n_{Q,i}} \right) - \frac{\bar{A}_j}{\bar{A}}, \\ F_{\text{ion},j} = \frac{k_B T}{m_u} \left(\frac{\Phi_j}{\bar{A}} - \frac{\Phi \bar{A}_j}{\bar{A}^2} \right).$$

Ion offsets use full-network number fractions:

$$F_{\text{off}} = \frac{\text{eV}}{m_u} \sum_i \tilde{y}_i I(Z_i), \quad F_{\text{off},j} = \frac{\text{eV}}{m_u} \sum_i \tilde{y}_{i,j} I(Z_i).$$

Effort: **M**. Main risk is keeping active-set and full-network normalizations distinct.

Electron term.

`compute_ideal_ele` evaluates the HELM free energy as

$$D = Y_e \rho, \quad F_e = Y_e f(T, D).$$

For

$$f_{a,b} = \frac{\partial^{a+b} f}{\partial T^a \partial D^b},$$

the stored coefficient is

$$\frac{\partial^{a+b} F_e}{\partial T^a \partial \rho^b} = Y_e^{b+1} f_{a,b}.$$

The composition derivative is

$$\frac{\partial}{\partial X_j} \left[\frac{\partial^{a+b} F_e}{\partial T^a \partial \rho^b} \right] = Y_{e,j} [(b+1) Y_e^b f_{a,b} + \rho Y_e^{b+1} f_{a,b+1}].$$

Current Skye overwrites HELM `xnefer` with complete-ionization `compute_xne`, so

$$\ln \text{free}_e = \ln Y_e, \quad \frac{\partial \ln \text{free}_e}{\partial X_j} = \frac{Y_{e,j}}{Y_e},$$

away from the existing $\max(10^{-99}, x_{\text{nefer}})$ clamp.

Effort: **M**. Most table derivatives already exist.

Coulomb term.

Treat Coulomb as a chain-rule problem over composition-dependent scalar inputs:

$$F_C = G(T, \rho, u_1, \dots, u_m), \quad F_{C,j} = \sum_a \frac{\partial G}{\partial u_a} \frac{\partial u_a}{\partial X_j}.$$

The implementation should use a Skye-local companion type carrying $(u, u_1, \dots, u_N)$, where each entry is an `auto_diff_real_2var_order3` in T, ρ . This avoids hand-expanding every Coulomb fit.

Effort: **L/XL**. This is the largest EOS-side piece because of liquid/solid fits, mixing entropy, phase softmin, and branch behavior.

Skye branch risks.

Derivatives are only smooth while the active set is unchanged:

$$X_i > \text{mass_fraction_limit_for_Skye}.$$

At exact active-set boundaries, min / max boundaries, and phase branch boundaries, use the derivative of the selected value branch and avoid those points in finite-difference tests.

eosDT and Blends

Current table mapping.

`eos/private/eosdt_eval.f90:get1_for_eosdt` maps table derivatives by charge:

$$\frac{\partial R}{\partial X_j} = \frac{\partial R}{\partial X_H} H_j + \frac{\partial R}{\partial Z} M_j,$$

with

$$H_j = \begin{cases} 1, & Z_j = 1, \\ 0, & \text{otherwise,} \end{cases} \quad M_j = \begin{cases} 1, & Z_j > 2, \\ 0, & \text{otherwise.} \end{cases}$$

Helium gets zero table derivative because the tables use $Y = 1 - X_H - Z$.

Blend derivative to add.

For

$$R = \alpha R_1 + (1 - \alpha) R_2,$$

the complete composition derivative is

$$R_j = \alpha R_{1,j} + (1 - \alpha) R_{2,j} + \alpha_j (R_1 - R_2).$$

Current `Do_Blend` has the first two terms only. Add α_j , including the existing quintic smoothing:

$$\alpha = h(a_0) = 10a_0^3 - 15a_0^4 + 6a_0^5,$$

$$\alpha_j = h'(a_0) a_{0,j}, \quad h'(a_0) = 30(a_0 - 1)^2 a_0^2.$$

Effort: **M**. The API plumbing is mechanical; alpha derivatives need case-by-case audit.

Star-Side Consumers

Current Ledoux/MHM approximation.

Current MESA computes the Brunt composition term in `star/private/brunt.f90:do_brunt_B_MHM_form` by calling the EOS twice at the same face (ρ_f, T_f) , but with neighboring cell compositions:

$$\ln P_1 = \ln P_{\text{eos}}(\rho_f, T_f, X_k), \quad \ln P_2 = \ln P_{\text{eos}}(\rho_f, T_f, X_{k-1}).$$

Then

$$B_{\text{MHM}} = \frac{\ln P_1 - \ln P_2}{(\ln P_{\text{eos},k-1} - \ln P_{\text{eos},k}) \chi_{T,f}}.$$

Linearizing the numerator gives

$$\ln P_1 - \ln P_2 \approx \sum_j \chi_{X_j,f} (X_{j,k} - X_{j,k-1}),$$

so

$$B_{\text{MHM}} \approx -\frac{1}{\chi_{T,f}} \sum_j \chi_{X_j,f} \frac{X_{j,k-1} - X_{j,k}}{\ln P_{\text{eos},k-1} - \ln P_{\text{eos},k}}.$$

That is the same first-order Ledoux chain-rule structure, but it gets the EOS composition contraction from a finite pressure difference. After `brunt_B` is computed, `hydro_vars.f90` stores it in `gradL_composition_term`, and `turb_support.f90` uses

$$\nabla_L = \nabla_{\text{ad}} + \text{gradL_composition_term}.$$

Brunt chain-rule form.

The branch `origin/Brunt_B_from_eos_partials` wants

$$B_{\text{comp}} = -\frac{1}{\chi_T} \sum_j \chi_{X_j} \frac{dX_j}{d \ln P}.$$

The EOS partial itself must be returned by the EOS at the face state:

$$g_{j,f} = \left. \frac{\partial \ln P_{\text{gas}}}{\partial X_j} \right|_{\rho_f, T_f, X_f}.$$

This is the part we are trying to fix. It should not be approximated by finite-differencing EOS calls with neighboring cell compositions. Since MESA currently returns gas-pressure partials, the total-pressure conversion must then be applied at the same face:

$$\chi_{X_j} = \frac{P_{\text{gas}}}{P_{\text{eos}}} g_{j,f},$$

$$\chi_T = \frac{P_{\text{gas}}}{P_{\text{eos}}} \left. \frac{\partial \ln P_{\text{gas}}}{\partial \ln T} \right|_{\rho, X} + \frac{4P_{\text{rad}}}{P_{\text{eos}}}.$$

The remaining finite difference is the spatial composition gradient, not an EOS composition partial:

$$\frac{dX_j}{d \ln P} \approx \frac{X_{j,k-1} - X_{j,k}}{\ln P_{\text{eos},k-1} - \ln P_{\text{eos},k}}.$$

Effort: **M**, after analytic EOS partials exist.

Implicit diffusion coupling.

Current element diffusion is implicit inside `star/private/diffusion.f90:do_solve_diffusion`, but it is operator-split from the hydro Newton solve. MESA already has one hydro composition equation per isotope in `star/private/hydro_chem_eqns.f90:do1_chem_eqns`; coupled microscopic diffusion should add a rate to that equation, not create a second species equation.

In current term names,

$$\begin{aligned} \text{dxdt_actual}_{j,k} &= \frac{X_{j,k}^{n+1} - X_{j,k}^n}{\Delta t}, & \text{dxdt_expected}_{j,k} &= \text{dxdt_mix}_{j,k} + \text{dxdt_nuc}_{j,k}, \\ s\%equ(i, k) = R_{j,k}^X &= \frac{\text{dxdt_expected}_{j,k} - \text{dxdt_actual}_{j,k}}{\text{eqn_scale}_{j,k}} = 0, & i &= s\%nvar_hydro + j. \end{aligned}$$

The coupled diffusion term should be

$$\begin{aligned} \text{dxdt_expected}_{j,k} &= \text{dxdt_mix}_{j,k} + \text{dxdt_nuc}_{j,k} + \text{dxdt_diff}_{j,k}, \\ \text{dxdt_diff}_{j,k} &= -\frac{\Phi_{j,k+1/2} - \Phi_{j,k-1/2}}{\Delta m_k}. \end{aligned}$$

To allow some transport to remain split while another part is coupled, accumulate the coefficient in two pieces:

$$D_{\text{mix}} = D_{\text{mix}}^{\text{explicit}} + D_{\text{mix}}^{\text{implicit}}.$$

Here explicit means outside the main hydro Newton block, or included as a frozen source with no new Jacobian. The implicit part supplies the new rate and Jacobian:

$$\text{dxdt_diff, implicit}_{j,k} = -\frac{\Phi_{j,k+1/2}^{\text{implicit}} - \Phi_{j,k-1/2}^{\text{implicit}}}{\Delta m_k}, \quad \frac{\partial R_{j,k}^X}{\partial X_{\ell,m}} \supset \frac{1}{\text{eqn_scale}_{j,k}} \frac{\partial \text{dxdt_diff, implicit}_{j,k}}{\partial X_{\ell,m}}.$$

The mass-fraction constraint can be handled by a sink species or by

$$R_k^\Sigma = \sum_j X_{j,k} - 1 = 0.$$

Effort: **XL**. EOS composition partials are prerequisite, not the whole implicit-diffusion implementation.

Rotation and other transport should stay opt-in.

Default behavior should not change: `do_element_diffusion` keeps using the current operator-split diffusion solver, and rotation/ordinary mixing keep entering through `s%dxdt_mix`. Rotation modifies `s%D_mix` through `star/private/mix_info.f90:update_rotation_mixing_info`:

$$D_{\text{mix}} = D_{\text{mix,nonrot}} + D_{\text{mix,rot}}.$$

A low-risk control structure is:

<code>do_element_diffusion</code>	current operator-split path
<code>use_coupled_element_diffusion</code>	new default-off Newton path
<code>use_coupled_diffusion_rotation_terms</code>	default off
<code>use_coupled_diffusion_other_D_mix</code>	default off

Internally:

$$\begin{aligned} D_{\text{mix,explicit}} &= D_{\text{mix,nonrot}} + D_{\text{mix,rot}} + \dots, \\ D_{\text{mix,implicit}} &= \text{coupled microscopic diffusion or selected coupled terms}, \\ D_{\text{mix}} &= D_{\text{mix,explicit}} + D_{\text{mix,implicit}}. \end{aligned}$$

The exact implementation may need more than scalar D_{mix} arrays for multicomponent diffusion, but the split keeps the interface clear: explicit/split terms keep the current path, implicit terms provide residual and Jacobian contributions. Any additional coupled transport source should return both its rate and Jacobian contribution. It should also conserve mass fraction in the solver basis:

$$\sum_j dxdt_diff_{j,k} = 0 \quad \text{or} \quad \sum_j \Phi_{j,k+1/2} = 0,$$

up to the chosen sink/projection convention.

Effort: **M** for controls and plumbing, larger if rotation's angular-momentum coupling is pulled into the same Newton block.

Verification Plan

Finite differences are tests, not production partials.

Raw EOS-partial check:

$$\left. \frac{\partial f}{\partial X_j} \right|_{\rho,T} \approx \frac{f(X_j + \epsilon) - f(X_j - \epsilon)}{2\epsilon}.$$

Constrained star-side check with sink species s :

$$D_j^{\text{sink}} f \approx \frac{f(X_j + \epsilon, X_s - \epsilon) - f(X_j - \epsilon, X_s + \epsilon)}{2\epsilon}.$$

Brunt contraction check:

$$B_{\text{comp}} = -\frac{1}{\chi_T} \sum_j \chi_{X_j} \frac{dX_j}{d \ln P}, \quad \sum_j \frac{dX_j}{d \ln P} \approx 0.$$

Run these only after explicit permission to compile/run MESA or targeted EOS tests.

Implementation Order

1. Add the composition helper and source-level tests of its formulas.
2. Add Skye ideal-ion and electron `d_dxa` for `i_lnPgas` and `i_lnE`.
3. Add a Skye-local composition companion for Coulomb and phase outputs.
4. Fill all internal Skye `d_dxa(1:nv,:)` rows, but keep the public `num_eos_d_dxa_results = 2` until the star-side API audit is explicit.
5. Add eosDT blend `dalpha/dX_i` plumbing and alpha derivatives.
6. Revisit the Brunt branch and remove the finite-difference fallback from production behavior.
7. Decide which diffusion and mixing terms belong in `D_mix_explicit` and which belong in `D_mix_implicit`.